SISTEMAS OPERATIVOS AVANZADOS

Sistemas de Archivos Modernos y CoW (Copy-on-Write)

Resiliencia, integridad estructural y arquitectura atómica en ZFS, Btrfs y almacenamiento de nueva generación.

Btrfs & ZFS Architecture

Protección Total de Datos

Escalabilidad Enterprise

Martinez Zarate Carlos

EVOLUCIÓN HISTÓRICA

De la Disquetera a los Petabytes

Era POSIX Tradicional

Diseñados para discos mecánicos pequeños con baja tasa de error y sin checksums integrados.

Introducción del Journaling

Sistemas como ext3/ext4 y NTFS agregaron diarios para acelerar la recuperación tras fallos.

La Revolución CoW

ZFS y Btrfs unifican volumen, RAID y sistema de archivos con garantía atómica de integridad.



ZFS (Sun Microsystems, 2005)

Diseño de 128 bits para capacidades virtualmente ilimitadas.



Btrfs (Oracle / Linux, 2007)

Integrado directamente en el kernel oficial de Linux.

EL PROBLEMA HISTÓRICO

Limitaciones de la Escritura In-Place

Escritura In-Place

Sobreescritura directa de bloques. Cortar la energía durante la modificación destruye la versión previa.

Chequeo Lento (fsck)

Tras un apagón inesperado, requiere escanear volúmenes completos durante horas antes del arranque.

Corrupción Silenciosa

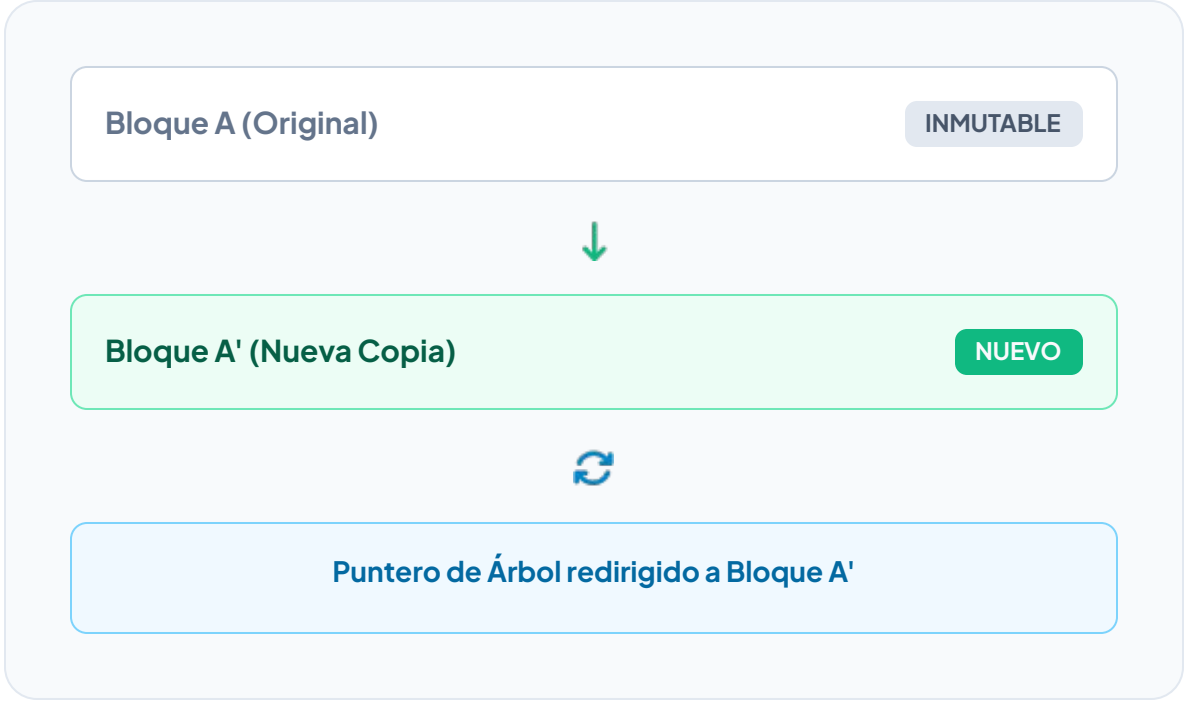
El «Bit Rot» degrada sectores físicos en silencio sin advertir al sistema operativo ni corregir errores.

EL NUEVO PARADIGMA

¿Qué es Copy-on-Write (CoW)?

Los datos existentes son inmutables y jamás se sobreescriben.

- **Escrituras en espacio libre:** Todo cambio asigna nuevos bloques de almacenamiento.
- **Actualización atómica:** Los punteros se redirigen solo al completar la operación.
- **Cero corrupción por cortes:** La versión previa permanece válida e intacta si hay fallos.



MECANISMO INTERNO

Anatomía de una Escritura CoW

1

Asignación

El SO localiza bloques no utilizados en el almacenamiento.

2

Escritura

Graba los datos modificados en las nuevas ubicaciones.

3

Commit Atómico

El árbol de metadatos actualiza la raíz de punteros.

4

Liberación

El bloque antiguo se marca libre o se preserva para snapshots.

CAPACIDAD CLAVE

Instantáneas (Snapshots) y Clones Eficientes

Creación O(1) Inmediata

Congela la raíz del árbol de metadatos en milisegundos sin copiar datos.

Cero Espacio Inicial

Un snapshot nuevo consume 0 KB y solo crece al modificar bloques.

Clones Escribibles

Permite bifurcar entornos de prueba al instante sin duplicar almacenamiento.



RESILIENCIA ABSOLUTA

Integridad de Datos y Autocuración (Self-Healing)



🧮 Checksums en Todos los Bloques

Hashes (XXHash, CRC32C, SHA-256) guardados en los bloques padre de metadatos.

Detección Inmediata de Errores

Al leer datos, el sistema calcula el checksum y lo compara con el metadato.

🪄 Reparación Transparente

En arreglos espejo o RAID, lee la réplica sana y corrige el bloque dañado al vuelo.

EFICIENCIA ESPACIAL

Compresión Transparente y Deduplicación

Compresión ZSTD / LZ4

Comprime bloques en RAM antes de escribir a disco de forma invisible al usuario.

Ahorro de 30% a 60%

Deduplicación de Datos

Consolida bloques idénticos apuntando al mismo checksum único en memoria.

Elimina duplicados

Ganancia de E/S (I/O)

Escribir menos datos físicos acelera el rendimiento general en discos HDD y SSD.

Menor desgaste SSD

BACKUPS DE ALTA EFICIENCIA

Replicación Incremental con Send / Receive

Transfiere la diferencia exacta entre dos snapshots a través de un flujo binario.

- Sin escaneo de archivos: Compara árboles de metadatos en memoria en lugar de escanear el disco con rsync.
- Ideal para red: Envía únicamente deltas comprimidos cifrados por SSH a servidores remotos.
- Preservación exacta: Mantiene checksums, permisos, propiedades y snapshots intactos.

```
# Replicación incremental en ZFS entre servidores
$ zfs send -i pool/datos@snap1 pool/datos@snap2 | \ ssh
backup-server zfs recv remote_pool/datos

# Envio de diferencial en Btrfs a disco externo
$ btrfs send -p /mnt/snap_ayer /mnt/snap_hoy | \ btrfs receive
/backup/externo
```

ESTRUCTURA DE DATOS

Arquitectura Interna: B-Trees vs. Merkle Trees

Btrfs: B-Trees Modificados

Utiliza árboles B de Copy-on-Write genéricos para administrar extensiones, inodos, subvolúmenes y asignación de bloques.

- Múltiples árboles especializados por función.
- Excelente flexibilidad para añadir discos heterogéneos.

ZFS: Merkle Trees Criptográficos

Árbol de datos donde cada nodo padre contiene el checksum validado de todos sus nodos hijos hasta la raíz (Uberblock).

- Verificación top-down en cada lectura.
- Inmunidad matemática contra corrupción silenciosa.

SISTEMAS EMBLEMÁTICOS

Btrfs vs. ZFS: Comparativa Técnica

Característica	Btrfs (Butter FS)	ZFS (Zettabyte FS)
Licencia y Kernel	GPL (Integrado nativamente en Linux)	CDDL (Módulo de kernel OpenZFS)
Estructura Principal	B-Trees adaptativos con subvolúmenes	Merkle Trees con zpools unificados
Gestión de RAID	A nivel de bloque / Tamaños mixtos	RAID-Z1/Z2/Z3 integrado al vdev
Uso Principal	Fedora, Synology NAS, Laptops, Workstations	Servidores Enterprise, FreeBSD, TrueNAS

CONSIDERACIONES TÉCNICAS

Desafíos y Compromisos de CoW

Fragmentación de Archivos

Bases de datos y máquinas virtuales con escritura aleatoria se fragmentan rápido. Requieren la opción nodatacow o desfragmentación.

Alto Consumo de RAM

ZFS ARC y las tablas de deduplicación demandan cantidades elevadas de memoria RAM (al menos 1 GB por TB de almacenamiento).

Rendimiento en Disco Lleno

Superar el 80%-85% de capacidad ralentiza la asignación de bloques libres al tener que buscar espacios dispersos.

DEMOSTRACIÓN PRÁCTICA

Comandos de Administración en Producción

```
// 1. Crear Pool ZFS RAID-Z2 de alto rendimiento con compresión ZSTD
$ sudo zpool create tank raidz2 /dev/sdb /dev/sdc /dev/sdd /dev/sde $ sudo zfs set compression=zstd tank

// 2. Crear snapshot y subvolumen en Btrfs
$ sudo btrfs subvolume create /mnt/datos/subvol1 $ sudo btrfs subvolume snapshot -r /mnt/datos/subvol1
/mnt/datos/snap_backup

// 3. Iniciar rutina de matorral (scrub) para verificar checksums en segundo plano
$ sudo zpool scrub tank # o bien: sudo btrfs scrub start /mnt/datos
```



¿Preguntas o Comentarios?

El almacenamiento moderno es inmutable, con verificación atómica de checksums e inmune a fallos eléctricos.

Sistemas de Archivos Modernos & Copy-on-Write

Image Sources



https://attom.tech/wp-content/uploads/2026/06/imgi_5_attom_ATTOM-Data-Center-Rack-System_20260517.webp

Source: attom.tech



https://www.getfulltextresearch.com/wp-content/uploads/2026/09/Emerald-usage.png

Source: www.getfulltextresearch.com